



Gymnázium J. K. Tyla

DarkFactory
Agentic and Harness Engineering:
Artificial Intelligence in Practice

THESIS

Author: Patrik Marius, 4.D
Supervisor: Michal Dočekal

Extent: 5358 words / 35840 characters

Declaration

I declare that I prepared this specialized thesis independently under the supervision of the supervisor named on the title page. All sources used are listed in the bibliography and information derived from them is cited in the text. I agree that the printed version may be archived at Gymnázium J. K. Tyla and used there as a reference for future student work or for presenting education at GJKT.

In Hradci Králové on

.....

Author signature:

Annotation

[EN] This thesis examines the principles of agentic engineering: effective engineering practices for development with artificial intelligence through agentic systems and the architecture of these systems. The practical contribution of the thesis is the design and implementation of DarkFactory — an agentic harness installable as a GitHub App. The system aims to maximize automation of the development lifecycle, from interpreting requirements in GitHub Issues, through planning, to code development and merging changes. The thesis reflects that current agentic systems cannot be regarded as fully autonomous: language models require deterministic guardrails, context management, and human involvement (*Human-in-the-loop*).

Keywords

, Agent Harness , Agent Loop , , , , , , Hook , LLM , Loop Engineering , MCP , Plugins , , Skills , Transformer

Contents

Declaration	2
Annotation	3
Keywords	4
1 Introduction	8
1.1 Motivation and Problem Definition	8
1.1.1 Global Generative AI Adoption	8
1.2 Thesis Objective and Research Questions	9
1.2.1 Main Goal	9
1.2.2 Sub-goals	9
1.2.3 Research Questions	9
1.3 Methodology	9
2 Development Environment and Practices	9
2.1 GitHub	10
2.2 Software Engineering	10
2.2.1 Planning	10
2.3 Version control	10
2.3.1 Git	10
2.3.1.1 Repository Branch	11
2.3.1.2 Branch Merge	11
2.3.1.3 Commit Squashing	11
2.3.1.4 GitHub Issue	12
2.3.1.5 Pull Request	12
2.3.1.6 Required Checks	12
2.3.1.7 Branch Protection	12
2.4 Continuous Integration	12
2.4.1 GitHub Actions	12
2.4.2 Software Container	12
2.4.3 Flaky Test	13
3 Language Models, Chatbots, and Agents	13
3.1 Chatbot	13
3.1.1 ChatGPT	13
3.1.1.1 ChatGPT App Interface	13
3.2 Agent	14
3.3 Large Language Model	14

3.3.1	GPT-5.6	15
3.3.1.1	GPT-5.6 Product Visual	15
3.3.2	Claude Opus 5	15
3.3.2.1	Claude Opus 5 Product Visual	15
3.3.3	DeepSeek-V4.1-Flash	16
3.3.3.1	DeepSeek-V4.1-Flash Product Visual	16
3.3.4	Transformer Architecture	17
3.3.5	Autoregressive Modeling	17
3.3.6	Stochastic Decoding	17
3.3.7	Tokenizer	17
3.3.8	Token	17
3.3.9	Embedding	17
3.3.10	Context Window	18
3.3.11	Key-Value Cache	18
3.3.12	Interaction Turn	18
4	Agentic Engineering	18
4.1	Prompt Engineering	19
4.1.1	System Prompt	19
4.2	Agent Harness	19
4.2.1	Codex	19
4.2.1.1	Codex App Interface	20
4.2.2	Claude Code	20
4.2.2.1	Claude Code Interface	20
4.2.3	Claude Desktop App	21
4.2.3.1	Claude Desktop Interface	21
4.2.4	Sandbox	22
4.2.5	Deterministic Guardrail	22
4.2.6	Human-in-the-loop	22
4.2.7	Plugins	23
4.2.8	ReAct Loop	23
4.2.8.1	Agent Session	24
4.2.8.2	Execution-loop Engineering	25
4.2.8.3	Agent Divergence Pathologies	25
4.2.9	Tool Calling	25
4.2.9.1	Model Context Protocol	25
4.2.10	Skills	26

4.2.10.1	Progressive Disclosure	26
4.2.10.2	Script	26
4.2.10.3	Event Hook	26
4.2.11	Context Engineering	27
4.2.11.1	Context Compaction	27
4.2.11.2	Retrieval-Augmented Generation	27
4.2.11.3	Context Rot	28
4.2.11.4	Semantic Drift	28
4.2.12	Workflow-graph Engineering	29
4.2.12.1	Subagent	29
4.2.12.2	Directed Acyclic Graph	29
5	Results and Discussion	29
6	Conclusion	29
	List of figures and tables	30
	List of appendices	31
	References	32

1 Introduction

1.1 Motivation and Problem Definition

Motivací práce je ukázat, jaké úlohy dokážou současné agentní systémy samostatně provádět a jaká technická vrstva umožňuje převést schopnost jazykového modelu do spolehlivého jednání nad skutečným softwarovým projektem. Generativní AI se během několika let rozšířila do masového používání. Microsoft AI Economy Institute odhaduje, že v prvním čtvrtletí 2026 použilo generativní AI 17,8 % světové populace v produktivním věku. Současně se rychle rozšiřují systémy, které už pouze negenerují odpověď, ale plánují více kroků, pracují se soubory a nástroji, spouštějí příkazy a testy, kontrolují vlastní výstupy a mohou vykonávat dlouhotrvající úlohy. ⁽¹⁾

V softwarovém inženýrství tento posun reprezentují například Codex a Claude Code: jejich oficiální popisy uvádějí práci nad reálnými repozitáři, paralelní agentní úlohy, úpravy souborů, spouštění testů a další činnosti přesahující jednorázové generování kódu. ^(2,3)

Práce proto nezkoumá pouze schopnost modelu vytvořit fragment programu. Zaměřuje se na **Agent Harness** jako systémovou vrstvu, která modelu poskytuje nástroje, stav, kontext, oprávnění, kontrolní smyčku a ověřování výsledků. Právě tato vrstva rozhoduje o tom, zda lze schopnosti modelu využít jako opakovatelný agentní proces.

1.1.1 Global Generative AI Adoption

Grafický podklad dokumentuje rychlost, s níž se generativní AI rozšířila z experimentální technologie do běžného používání. Microsoft AI Economy Institute uvádí pro druhé pololetí roku 2025 celosvětový podíl uživatelů 16,3 %, oproti 15,1 % v prvním pololetí. Novější zpráva pro první čtvrtletí 2026 odhaduje používání generativní AI na 17,8 % světové populace v produktivním věku.

Region	H1 2025 AI Diffusion	H2 2025 AI Diffusion	Change
Global North	22.9%	24.7%	+1.8
Global South	13.1%	14.1%	+1.0
World	15.1%	16.3%	+1.2

Obrázek 1: AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute.

Měřítka adopce neposuzuje schopnosti jednotlivých systémů, ale dokládá jejich rychlé rozšíření do reálného používání. ^(1,4) Výchozí otázkou práce je, co současné agentní systémy skutečně dokážou a jak musí být navržen harness, aby jejich schopnosti bylo možné bezpečně a opakovatelně využít v reálném vývojovém prostředí. ^(1–3)

1.2 Thesis Objective and Research Questions

1.2.1 Main Goal

Vymezit teoretické principy agentického inženýrství (*agentic engineering*) a navrhnout modulární architekturu agent harnessu pro automatizovaný vývoj softwaru se zachováním lidského dohledu v klíčových rozhodovacích bodech.

1.2.2 Sub-goals

- Vymezit infrastrukturu pro správu verzí (Git, GitHub a kontinuální integraci).
- Analyzovat limity velkých jazykových modelů (dynamiku kontextového okna, jev Context Rot, ztrátovou kompresi a sémantický posun).
- Navrhnout architekturu agent harnessu zahrnující nástrojové smyčky (ReAct), bezpečnostní pískoviště a hierarchickou orchestraci subagentů.
- Formalizovat mechanismy zapojení člověka do smyčky (*Human-in-the-loop*), schvalovací brány a protokol revizních značek pro dohled nad textovými výstupy.

1.2.3 Research Questions

1.3 Methodology

Práce používá konceptově-analytický a inženýrský postup: vymezuje nezbytné části agentního systému, popisuje jejich vztahy a ověřuje je na návrhu a realizaci systému DarkFactory. Předmětem práce není trénování neuronových sítí, optimalizace vah ani podrobná matematika modelového učení. **LLM** je zde chápán jako hotová inferenční komponenta. Modelová vrstva je proto popsána pouze v rozsahu nutném pro pochopení agentního běhu, tokenizace, kontextového okna, paměťového stavu inference a nástrojových rozhraní.

Hlavním předmětem zkoumání je a konstrukce **Agent Harness** pro autonomní softwarové inženýrství. Jednotlivé mechanismy jsou rozděleny do samostatných konceptů, aby každý pojem měl jedno místo pro definici, popis a shrnutí a aby se stejné vysvětlení neopakovalo v několika kapitolách.

Postup práce má tři kroky: nejprve jsou vymezeny relevantní koncepty a jejich závislosti; následně jsou tyto principy promítnuty do architektury DarkFactory; nakonec jsou vlastnosti výsledného systému posouzeny podle ověřitelných výstupů vývojového procesu. Terminologie zachovává zavedené oborové názvy tam, kde jsou v praxi přesnější než doslovný překlad, přičemž všechny používané termíny jsou vedeny jako kanonické koncepty. Rozsah práce je záměrně soustředěn na agentní systém kolem modelu: model je vstupní inferenční komponenta, zatímco předmětem návrhu a hodnocení je harness a jeho provozní mechanismy.

2 Development Environment and Practices

Vývojové prostředí a praxe tvoří soubor verzovacích, plánovacích, integračních a kontrolních postupů, které poskytují deterministický rámec pro agentní vývoj softwaru. Agentní

systém nepracuje pouze s textem, ale se stavem repozitáře a vývojovým procesem. Proto musí být jeho akce ukotveny v explicitních postupech pro verzování, plánování změn, automatické kontroly a revizi výsledků. Deterministické vývojové prostředí převádí jednotlivé agentní akce na dohledatelný a ověřitelný softwarově-inženýrský proces.

2.1 GitHub

GitHub je cloudová platforma pro hosting gitových repozitářů a koordinaci vývojového cyklu pomocí Issues, Pull Requests a automatizačních workflow. GitHub zde tvoří koordinační vrstvu nad Gitem, která propojuje zadání, revizi změn a automatizované kontroly.

2.2 Software Engineering

Softwarové inženýrství je systematické uplatňování inženýrských principů na specifikaci, návrh, implementaci, ověřování, provoz a údržbu softwarových systémů. Pro agentní systémy je tento rámec důležitý proto, že generování kódu představuje pouze jednu část životního cyklu změny. Výsledek musí být zasazen do specifikace požadavku, řízeného procesu změn, automatického ověřování a následné revize. Agentní vývoj je podmnožinou softwarového inženýrství: kvalita výsledku se neposuzuje podle množství vygenerovaného kódu, ale podle splnění požadavků a ověřitelnosti změny.

2.2.1 Planning

Plánování je proces převodu požadavku na explicitní posloupnost kroků, závislostí a ověřovacích podmínek před prováděním změn. V agentním vývoji plán omezuje okamžité přecházení od požadavku k úpravám kódu. Rozděluje práci na kontrolovatelné kroky a předem určuje, jak bude možné ověřit, že jednotlivé části i celek splnily zadání. Plánování vytváří kontrolovatelný mezistupeň mezi požadavkem a provedením a snižuje riziko, že agent optimalizuje lokální změny bez ohledu na celek.

2.3 Version control

Správa verzí je řízení a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a vracet. Pro autonomní vývoj poskytuje verzovací vrstva deterministický záznam reality, který je nezávislý na interním kontextu modelu. Agent může pracovat nad izolovanou změnou, její stav porovnat s výchozí verzí a v případě neúspěchu se vrátit k známému bodu historie. Správa verzí je základní stavová a návratová vrstva agentního vývoje; model navrhuje změny, ale repozitář uchovává jejich skutečnou historii.

2.3.1 Git

Git je distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře. Pro autonomní vývoj softwaru je spolehlivá správa verzí naprosto nezbytným základem.

Jazykové modely generují kód na základě statistické pravděpodobnosti, a proto se nevyhnutelně dopouštějí chyb, logických přehmatů či regresí. Verzovací systém vytváří bezpečné a deterministické prostředí, v němž lze každou úpravu zaznamenat, otestovat a v případě selhání kdykoliv vrátit zpět k funkčnímu stavu. Namísto teoretických abstrakcí práce přímo využívá distribuovaný systém ⁷ v kombinaci s platformou ⁸.

Klíčové komponenty infrastruktury zahrnují:

- Distribuovaný systém Git ⁽⁵⁾: Ukládá kompletní historii projektu v podobě jednotlivých revizí (*commitů*). Vývojář i agent pracují s plnou lokální kopií repozitáře, což umožňuje provádět změny, přepínat větve a spouštět lokální testy zcela nezávisle na síťovém připojení.
- Platforma GitHub: Slouží jako centrální bod pro sdílení kódu, týmovou koordinaci a automatizaci:
 - Zadávání a sledování úkolů (Issues): Strukturovaná textová zadání požadavků a hlášení chyb, která agentovi slouží jako výchozí specifikace úlohy.
 - Revize změn (Pull Requests): Uživatelské rozhraní pro přehledné zobrazení diffu, diskusi nad kódem a formální schválení člověkem.
 - Automatizace (GitHub Actions): Běhové prostředí pro automatické spouštění testů, linterů a překladů při každé události v repozitáři.

Git poskytuje agentnímu vývoji auditovatelnou historii a možnost bezpečně izolovat, porovnávat, slučovat nebo vracet změny bez závislosti na paměti modelu.

2.3.1.1 Repository Branch

Větev je pojmenovaná vývojová linie v systému správy verzí, která umožňuje provádět změny odděleně od jiné linie historie a později je porovnat nebo sloučit. Samostatná pracovní větev izoluje mezistavy a chyby agentního běhu od stabilní hlavní linie a umožňuje celý neúspěšný pokus bezpečně zahodit.

2.3.1.2 Branch Merge

Merge je operace správy verzí, která kombinuje změny nebo historii dvou vývojových linií do společného výsledného stavu; případné konflikty vyžadují explicitní vyřešení. Způsob sloučení určuje, jak se pracovní historie agenta promítne do stabilní větve; před integrací je vhodné oddělit užitečný výsledný stav od experimentálních mezikroků.

2.3.1.3 Commit Squashing

Squash je operace, při níž se více po sobě jdoucích *commitů* nahradí jedním souhrnným *commitem*, obvykle za účelem zjednodušení historie před integrací změn. V agentním běhu často vzniká více pomocných *commitů* během iterativního ladění. Před začleněním výsledné změny lze tyto mezikroky sloučit do jednoho logického záznamu odpovídajícího dokončenému úkolu. Squash odděluje experimentální průběh práce od dlouhodobé historie projektu a umožňuje uchovat jednu logickou změnu jako jeden auditovatelný *commit*.

2.3.1.4 GitHub Issue

GitHub Issue je strukturovaný záznam požadavku, úkolu nebo chyby v repozitáři, který může nést popis, diskusi, štítky, přiřazení a vazby na změny kódu. Issue funguje jako stabilní zdroj zadání oddělený od implementace a umožňuje pozdější plán, změny i revizi vztáhnout ke stejnému původnímu požadavku.

2.3.1.5 Pull Request

Pull Request je formální návrh na začlenění změn z jedné větve repozitáře do druhé a společné místo pro automatizované kontroly, lidskou revizi a diskusi nad navrženými úpravami. Pull Request vytváří kontrolní hranici mezi samostatně připravenou změnou a jejím přijetím do hlavní historie projektu.

2.3.1.6 Required Checks

Požadované kontroly jsou automatizované kontroly, jejichž úspěšné dokončení je povinnou podmínkou pro přijetí nebo sloučení změny. Required checks mění CI z informativní zpětné vazby na technicky vynucovanou podmínku integrace.

2.3.1.7 Branch Protection

Ochrana větví je sada pravidel repozitáře, která omezuje přímé změny chráněných větví a vynucuje schválení, kontroly nebo jiné podmínky před sloučením. GitHub poskytuje pravidla ochrany větví (*Branch Protection Rules*), která zabráňují začlenění neověřeného kódu do stabilní větve `main`.

- Povinné schválení člověkem: Požadavek na explicitní autorizaci kódu lidským vývojářem dříve, než GitHub povolí sloučení do produkční větve.

Ochrana větví převádí procesní pravidla integrace do technicky vynucované bariéry, kterou agent ani člověk nemůže obejít běžným přímým zápisem.

2.4 Continuous Integration

Průběžná integrace (CI) je vývojová praxe, při níž se změny často integrují a automaticky ověřují sestavením, testy a dalšími kontrolami.

V agentním vývoji CI poskytuje strojově ověřitelnou zpětnou vazbu, která odděluje generování změny od jejího objektivního ověření.

2.4.1 GitHub Actions

GitHub Actions je automatizační platforma GitHubu pro spouštění deklarovaných workflow a jejich jobů v reakci na události repozitáře nebo ruční spuštění. V této práci GitHub Actions představuje konkrétní prováděcí prostředí CI a repozitářové automatizace, nikoli samotný princip průběžné integrace.

2.4.2 Software Container

Softwarový kontejner je izolované uživatelské běhové prostředí, které balí aplikaci a její závislosti při sdílení jádra hostitelského operačního systému. Kontejner zvyšuje repro-

dukovatelnost běhu, ale jeho bezpečnostní izolace závisí na konkrétní implementaci a konfiguraci.

2.4.3 Flaky Test

Nestálý test je test, který může nad stejným kódem střídavě projít a selhat kvůli nedeterminismu, časování, prostředí nebo externím službám. Nestálé testy snižují informační hodnotu CI a mohou agenta vést k opravám správného kódu; podezřelé selhání proto musí být reprodukovatelné nebo explicitně označené jako nedeterministické.

3 Language Models, Chatbots, and Agents

Tato skupina konceptů rozlišuje samotný jazykový model od produktových rozhraní a agentních systémů, které model používají. **LLM**, **·** a **·** označují různé systémové vrstvy. Toto rozlišení zabraňuje zaměňování schopností modelu s funkcemi, které ve skutečnosti poskytuje aplikační nebo agentní harness. Další sekce proto popisují model, jeho bezprostřední inferenční mechanismy a nadřazené systémy odděleně.

3.1 Chatbot

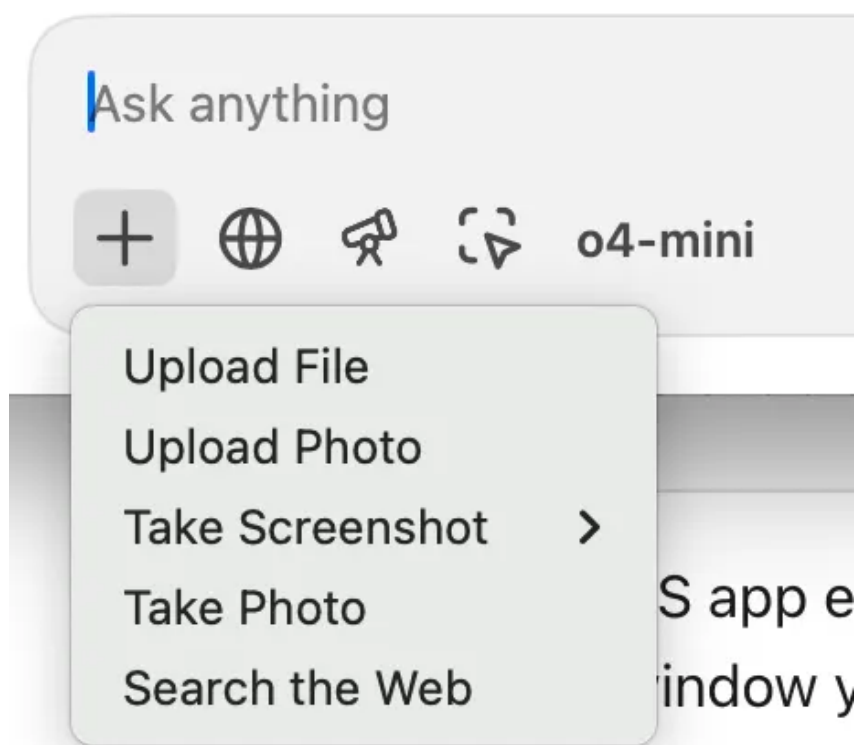
Chatbot je aplikační systém, který zpřístupňuje jazykový model prostřednictvím konverzační interakce. Na rozdíl od samotného modelu může chatbot přidávat historii konverzace, multimodální vstupy, práci se soubory, webové vyhledávání nebo další nástroje. Tyto funkce patří aplikační vrstvě, nikoli modelu samotnému.

3.1.1 ChatGPT

ChatGPT je konverzační produkt OpenAI zpřístupňující modely prostřednictvím chatového a aplikačního rozhraní. Jako příklad chatbota odděluje modelovou vrstvu od uživatelského produktu: rozhraní přidává práci se soubory, obrazem, webem a dalšími nástroji, které samotný model neposkytuje.

3.1.1.1 ChatGPT App Interface

Oficiální snímek rozhraní aplikace ChatGPT pro macOS. Snímek ukazuje vstupní rozhraní a nabídku práce se soubory, obrazem a webem.



Obrázek 2: Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.

Příklad uživatelského povrchu nad jazykovým modelem.⁽⁶⁾ ChatGPT ilustruje rozdíl mezi jazykovým modelem a aplikačním systémem, který model obaluje.⁽⁶⁾ Chatbot je produktová vrstva nad modelem; agentní systém se od něj odlišuje především řízenou prováděcí smyčkou a schopností samostatně vykonávat akce.

3.2 Agent

Agent je softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně vnímá stav prostředí a provádí víceukrokové akce směřující k zadanému cíli. Rozdíl mezi jazykovým modelem a agentem nespočívá nutně v použitém jazykovém modelu, ale v architektuře jeho zapojení do prostředí. Agent může číst stav repozitáře, provádět změny, spouštět nástroje a podle jejich výsledků pokračovat v další iteraci. Tuto schopnost zajišťuje nadřazená prováděcí a nástrojová vrstva, nikoli samotný model. Agent rozšiřuje model o řízené víceukrokové jednání nad skutečným prostředím; autonomii proto určuje celý agentní systém, ne pouze model.

3.3 Large Language Model

Velký jazykový model je neuronový model pro zpracování a generování posloupností tokenů; současné modely tohoto typu typicky používají **Transformer**. Při generování model opakovaně odhaduje další token na základě dosavadního kontextu. Samostatné koncepty dále popisují tokenizaci, reprezentaci dat a omezení kontextu. Funkce jako práce se sou-

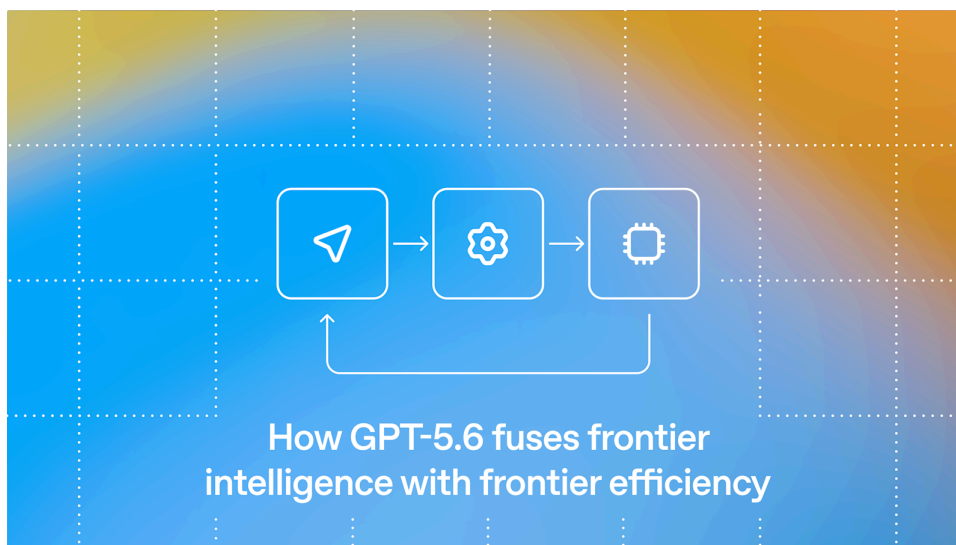
bory, spouštění příkazů nebo správa úloh nejsou vlastnostmi jazykového modelu; poskytuje je nadřazený aplikační nebo agentní systém.

3.3.1 GPT-5.6

GPT-5.6 je modelová řada OpenAI vydaná v roce 2026 ve variantách Sol, Terra a Luna. Jde o příklad současného jazykového modelu, který je nasazován jako inferenční komponenta v produktech včetně ChatGPT a Codexu; aplikační schopnosti těchto produktů však vznikají až propojením modelu s jejich harnessy.

3.3.1.1 GPT-5.6 Product Visual

Oficiální produktový vizuál modelové řady GPT-5.6. Obrazový podklad je převzat z oficiálního vydání modelu OpenAI.



Obrázek 3: GPT-5.6. Zdroj: OpenAI.

Vizuální identifikace příkladu jazykového modelu.⁽⁷⁾ Příklad odděluje modelovou vrstvu od agentního systému, který model používá.⁽⁷⁾

3.3.2 Claude Opus 5

Claude Opus 5 je model společnosti Anthropic vydaný v červenci 2026. Model je příkladem současné modelové vrstvy používané v širších agentních produktech. Samotný model není totožný s Claude Code ani desktopovou aplikací Claude; ty přidávají nástroje, stav a orchestrace.

3.3.2.1 Claude Opus 5 Product Visual

Oficiální produktový vizuál modelu Claude Opus 5. Obrazový podklad je převzat z oficiálního oznámení společnosti Anthropic.



Obrázek 4: Claude Opus 5. Zdroj: Anthropic.

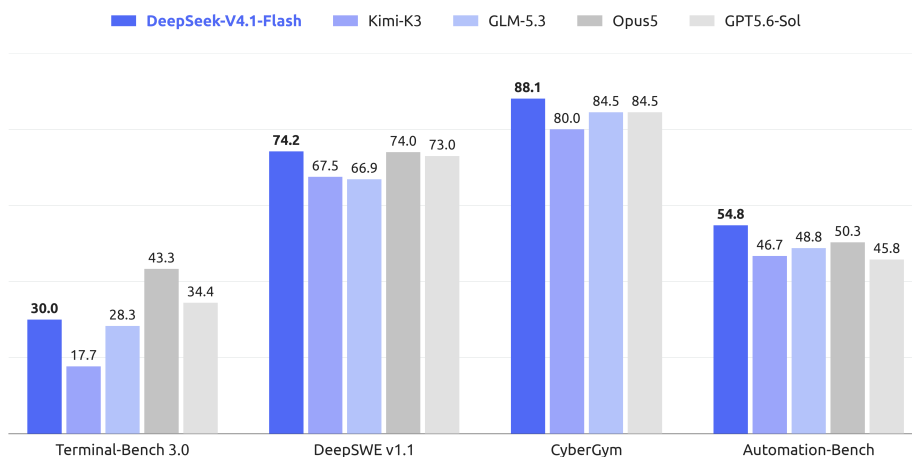
Vizuální identifikace příkladu jazykového modelu.⁽⁸⁾ Příklad zdůrazňuje rozdíl mezi modelem a agentním harness-em.⁽⁸⁾

3.3.3 DeepSeek-V4.1-Flash

DeepSeek-V4.1-Flash je multimodální model DeepSeek vydaný v září 2026. Jde o příklad odlišné současné modelové architektury: DeepSeek jej popisuje jako 552B MoE s asymetrickým Causal Encoder–Decoder uspořádáním a nativním vizuálním vstupem.

3.3.3.1 DeepSeek-V4.1-Flash Product Visual

Oficiální produktový vizuál modelu DeepSeek-V4.1-Flash. Obrazový podklad je převzat z oficiálního vydání společnosti DeepSeek.



Obrázek 5: DeepSeek-V4.1-Flash. Zdroj: DeepSeek.

Vizuální identifikace příkladu jazykového modelu.⁽⁹⁾ Příklad ukazuje, že pojem jazykového modelu v agentním systému zahrnuje různé konkrétní modelové rodiny a architektury.⁽⁹⁾ Jazykový model je inferenční komponenta. V této práci je důležité především jeho rozhraní a omezení, protože provozní chování agenta vzniká až propojením modelu s harness-em.

3.3.4 Transformer Architecture

Transformer je architektura neuronových sítí založená na mechanismu pozornosti, která modeluje vztahy mezi prvky sekvence a tvoří základ většiny současných velkých jazykových modelů. Pro tuto práci je podstatné, že transformer zpracovává tokenové reprezentace v omezeném kontextu a vytváří stav potřebný k postupnému odhadu dalších tokenů; detailní matematika trénování není předmětem práce.

3.3.5 Autoregressive Modeling

Autoregresivní modelování je sekvenční postup, při němž model odhaduje následující token podmíněně na předchozí tokeny v aktuální posloupnosti. Autoregrese vysvětluje, proč generování probíhá po jednotlivých krocích a proč každé nové pokračování závisí na dosavadním kontextu.

3.3.6 Stochastic Decoding

Stochastické dekódování je výběr výstupních tokenů z pravděpodobnostního rozdělení modelu způsobem, který může při stejném vstupu vést k různým platným pokračováním. Nedeterminismus dekódování je jedním z důvodů, proč kritická provozní pravidla agentního systému musí vynucovat harness mimo model.

3.3.7 Tokenizer

Tokenizer je komponenta, která převádí text nebo jiný podporovaný vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekódování. Konkrétní tokenizer určuje slovník i pravidla rozdělení vstupu, často pomocí subword metod, jako je Byte Pair Encoding. Stejný text proto může mít u různých modelových rodin odlišný počet tokenů a jinou segmentaci. Tokenizer je deterministické rozhraní mezi textovým povrchem a diskrétní sekvencí, kterou model skutečně zpracovává.

3.3.8 Token

Token je diskrétní jednotka zpracovávaná jazykovým modelem, odpovídající položce slovníku tokenizéru a reprezentovaná číselným identifikátorem. Token nemusí odpovídat celému slovu; podle použitého tokenizéru může představovat slovo, část slova, znakový fragment nebo jinou jednotku. Po tokenizaci jsou identifikátory převedeny na vektorové reprezentace, se kterými pracují vrstvy modelu. Token je základní diskrétní jednotka modelového vstupu a výstupu a současně jednotka, podle níž se prakticky měří velikost kontextu.

3.3.9 Embedding

Embedding je vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné vztahy mezi reprezentovanými objekty. V jazykovém modelu embedding převádí diskrétní identifikátory tokenů na spojitě vektory zpracovatelné neuronovou sítí. Podobné vektorové reprezentace lze použít také mimo

samotnou generaci, například pro vyhledávání semanticky podobných položek v systému RAG. Embedding propojuje diskrétní vstupy s numerickým prostorem modelu a může současně sloužit jako reprezentace pro sémantické vyhledávání.

3.3.10 Context Window

Kontextové okno je maximální rozsah tokenové sekvence, kterou model při jednom inferenčním běhu dokáže zahrnout do aktivního kontextu. Do tohoto limitu se společně započítávají systémové instrukce, uživatelský vstup, historie běhu, výsledky nástrojů a další data předaná modelu. Omezení proto přímo ovlivňuje, kolik pracovního stavu může agent udržovat současně bez výběru, externí paměti nebo kompakce. Kontextové okno je konečný pracovní prostor inference; harness musí aktivně rozhodovat, které informace v něm zůstanou.

3.3.11 Key–Value Cache

KV Cache je mezipaměť dříve vypočtených vektorů klíčů a hodnot v pozornostních vrstvách transformera, která při autoregresivním generování omezuje opakovaný výpočet předchozích tokenů. Při každém novém tokenu lze znovu použít klíče a hodnoty vytvořené pro předchozí část sekvence namísto jejich úplného přepočítání. Mezipaměť tím snižuje výpočetní režii generování, ale její velikost roste s délkou aktivní sekvence a představuje významnou část paměťových nároků inference. KV Cache urychluje autoregresivní inferenci výměnou výpočetní práce za paměť, jejíž spotřeba roste s kontextem.

3.3.12 Interaction Turn

Tah je jedna diskrétní jednotka interakce v konverzačním nebo agentním protokolu, například zpráva uživatele, výstup modelu nebo samostatně evidovaný výsledek nástroje. Agentní běh skládá sekvenci tahů, v níž se střídají vstupy prostředí, modelová rozhodnutí a výsledky provedených akcí. Historie těchto tahů tvoří část pracovního kontextu a harness rozhoduje, které z nich zůstávají modelu dostupné v dalších iteracích. Tah je protokolová jednotka agentní interakce; odděluje jednotlivé vstupy, rozhodnutí a pozorování, z nichž harness sestavuje další kontext.

4 Agentic Engineering

Agentické inženýrství je disciplína zaměřená na návrh, orchestraci a provoz agentních systémů kolem jazykových modelů, včetně nástrojů, kontextu, prováděcích smyček, mantinelů a lidského dohledu. Předmětem agentického inženýrství není samotná modelová architektura, ale systémové prostředí, které převádí modelový výstup na řízené jednání. Patří sem zejména návrh agentního harnessu, správa kontextu a stavu, rozhraní nástrojů, prováděcí smyčky, ověřování výsledků, izolace a rozhodovací body pro lidský dohled. Agentické inže-

nýrství hodnotí celý systém kolem modelu podle toho, zda dokáže modelové schopnosti převést na opakovatelné, kontrolovatelné a ověřitelné chování.

4.1 Prompt Engineering

An engineering discipline for systematically designing, structuring, and optimizing instructions and system prompts to guide and constrain language-model behavior. ⁽¹⁰⁾ Základní chování agenta vymezuje systémový prompt ⁽¹⁰⁾, který definuje jeho identitu, sadu dostupných nástrojů a provozní mantinely. Při formulaci těchto pravidel však vývojáři narážejí na specifickou vlastnost autoregresivních modelů — problematické zpracování zákazů a negativních instrukcí.

Příčiny a inženýrská řešení tohoto jevu:

- Úskalí negativních instrukcí: Zákazy formulované negací (např. „nemazat existující testy“) modely často porušují, protože matice pozornosti (QK^T) asociativně aktivuje zakázaný pojem dříve, než autoregresní proces uplatní logický operátor negace.
- Afirmativní formulace: Pravidla je nutné formulovat pozitivně — namísto výčtu zákazů vymezit přesný postup a povolené mantinely chování.
- Deterministická ochrana v harnessu: Kde nestačí prompt, musí zasáhnout kód agent harnessu — například zpřístupněním testovacích souborů pouze pro čtení nebo zablokováním destruktivních operací na úrovni systémového volání.

4.1.1 System Prompt

A high-priority instruction layer that defines the role, behavior, available capabilities, and operating constraints of a language model or agent. ⁽¹⁰⁾

4.2 Agent Harness

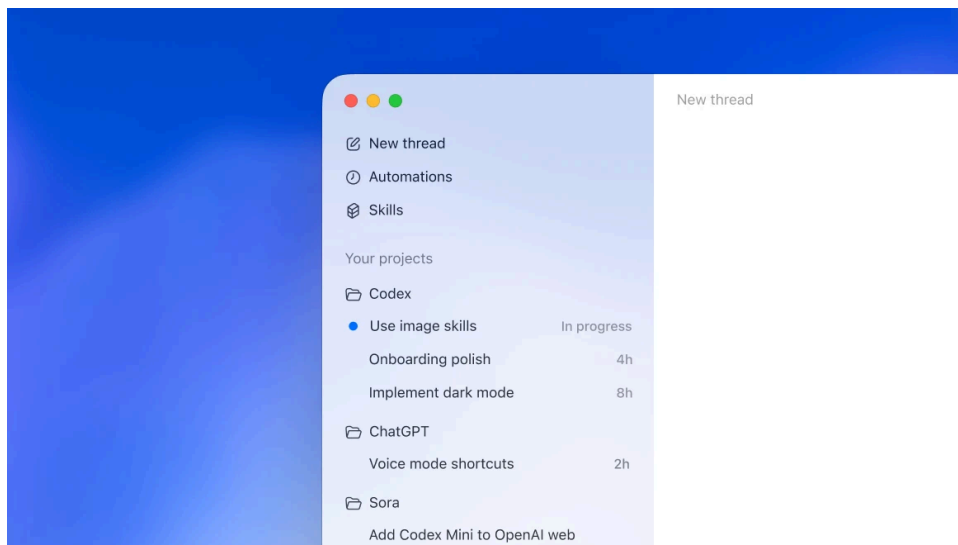
Agentní harness je aplikační a orchestrační vrstva, která propojuje model s prostředím a řídí jeho opakované jednání nad stavem a nástroji. Harness sestavuje kontext, zpřístupňuje a omezuje nástroje, spravuje stav sezení, vyhodnocuje výsledky akcí a rozhoduje o pokračování nebo ukončení prováděcí smyčky. Tím odděluje pravděpodobnostní rozhodování modelu od deterministických pravidel systému. Ústředním prováděcím mechanismem je **Agent Loop**.

4.2.1 Codex

Codex je agentní vývojový systém OpenAI určený pro samostatné plnění softwarově-inženýrských úloh. Desktopová aplikace umožňuje řídit více agentů paralelně, oddělovat jejich práci do vláken a worktree, kontrolovat diffy a delegovat dlouhotrvající úlohy. Tím představuje praktický příklad harnessu, který rozšiřuje model o stav, nástroje, izolaci a pracovní postup.

4.2.1.1 Codex App Interface

Oficiální produktový snímek aplikace Codex. Snímek dokumentuje specializované rozhraní pro práci s agentními úlohami a dovednostmi.



Obrázek 6: Aplikace Codex. Zdroj: OpenAI.

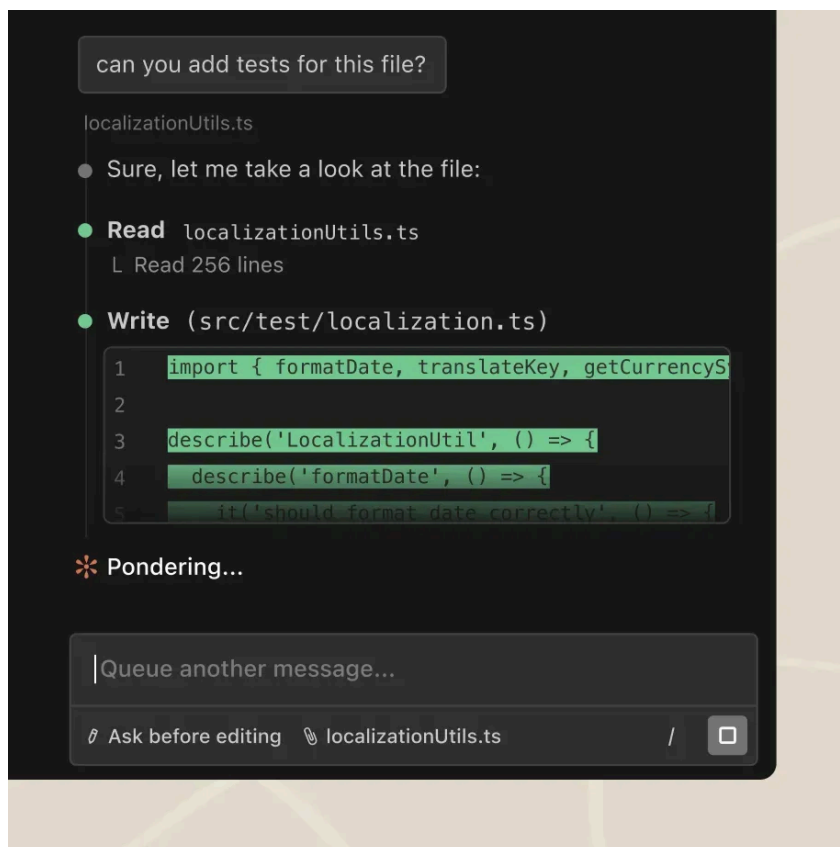
Vizuální příklad agentního vývojového prostředí.⁽²⁾ Codex ukazuje posun od generování fragmentů k řízenému agentnímu vývojovému procesu.⁽²⁾

4.2.2 Claude Code

Claude Code je agentní vývojové prostředí společnosti Anthropic dostupné v terminálu, IDE, na webu a v desktopové aplikaci. Systém může plánovat změny, číst a zapisovat soubory, spouštět příkazy a testy, používat Git a MCP a vytvářet pull requesty. Tyto schopnosti vznikají kombinací modelu s nástrojovou a stavovou vrstvou harnessu.

4.2.2.1 Claude Code Interface

Oficiální snímek rozhraní Claude Code při práci se zdrojovým kódem. Snímek zobrazuje čtení souboru, zápis změny a průběžný stav agentní úlohy.



Obrázek 7: Claude Code při práci se souborem. Zdroj: Anthropic.

Vizuální příklad nástrojově řízeného kódovacího agenta.⁽³⁾ Claude Code je konkrétní příklad agentního harnessu propojeného s reálným vývojovým prostředím.⁽³⁾

4.2.3 Claude Desktop App

Claude Desktop je desktopová aplikační vrstva nad modely Claude, která propojuje konverzaci s lokálními soubory, aplikacemi, webem a dalšími schopnostmi. Anthropic popisuje desktopovou aplikaci jako prostředí, v němž může Claude se souhlasem uživatele přímo číst, upravovat a ukládat soubory a pracovat s dalšími aplikacemi. Jde proto o širší příklad harnessu než samotný chatbot.

4.2.3.1 Claude Desktop Interface

Oficiální snímek aplikačního rozhraní Claude s pracovním artefaktem. Snímek ukazuje, že aplikační vrstva může vedle konverzace zpřístupnit soubory, nástroje a samostatný pracovní povrch.



Obrázek 8: Aplikační rozhraní Claude. Zdroj: Anthropic.

Vizuální příklad desktopového aplikačního harnessu.⁽¹¹⁾ Desktopová aplikace ukazuje, jak harness rozšiřuje model o oprávnění, kontext a akce v uživatelském prostředí.⁽¹¹⁾ Schopnosti agentního systému proto nelze připsat pouze modelu: vznikají souhrou modelu a harnessu, který mu poskytuje prostředí pro řízené jednání.

4.2.4 Sandbox

Sandbox je omezené běhové prostředí určené k oddělení prováděného kódu a jeho oprávnění od hostitelského systému. Přímé spouštění kódu (*Code Execution*) umožňuje agentovi generovat skripty (bash, Python), které harness spouští v izolovaném terminálu. Tento model poskytuje maximální flexibilitu pro softwarový vývoj, avšak vyžaduje nekompromisní bezpečnostní izolaci.

Bezpečné spouštění modelově generovaného kódu vyžaduje vynutitelnou izolaci procesů, souborů, oprávnění, tajností a sítě; běžný kontejner sám o sobě nemusí tvořit dostatečnou bezpečnostní hranici.

4.2.5 Deterministic Guardrail

Guardrail je programově vynucené omezení nebo kontrola, která neponechává kritické provozní pravidlo pouze na pravděpodobnostním rozhodnutí modelu. Kritická bezpečnostní a procesní pravidla patří do deterministicky vynucované vrstvy harnessu, nikoli pouze do textových instrukcí modelu.

4.2.6 Human-in-the-loop

Zapojení člověka do smyčky (HITL) je návrhový vzor, v němž lidský operátor zůstává součástí rozhodovacího procesu prostřednictvím explicitních schvalovacích bran, zejména před významnými nebo nevratnými operacemi. Základním principem navrženého řešení není nekritická plná autonomie, nýbrž efektivní kooperace člověka a stroje. Autonomnímu

systému náleží mechanické a rutinní úkony, zatímco klíčová architektonická a nevratná rozhodnutí zůstávají plně pod kontrolou vývojáře.

Řízení lidského dohledu staví na těchto pilířích:

- Lidské schvalovací brány (*Human Gates*): Formální procesní uzly, v nichž se automatický běh pozastaví a vyčká na autorizaci operátora:
 - 1. brána (Záměr a plán): Člověk autorizuje technický plán a rozpad požadavku dříve, než agent začne modifikovat kód v souborech.
 - 2. brána (Sémantická revize): Člověk provádí finální kontrolu diffu v pull requestu před jeho začleněním do hlavní větve.
- Prevence únavy z revizí (*Review Fatigue*): Vyvážená frekvence kontrol — zamezení mikromanagementu na úrovni jednotlivých souborů při zachování kontroly nad celkovým architektonickým směrem.
- Dohledatelnost původního zadání: Trvalé uchovávání doslovného znění požadavku (GitHub Issue) bez ztrátových parafrází modelem, což brání vymizení okrajových podmínek v průběhu vývoje.
- Transparentnost selhání a deterministická eskalace: Zákaz tichého pohlcování chyb či halucinovaných omluv při selhání. Při vyčerpání rozpočtu nebo selhání testů harness vygeneruje strukturovaný diagnostický incident (diff, chybové hlášení, stav kontextu) a předá jej vývojáři k manuálnímu zásahu.

Lidský dohled je nejúčinnější v několika explicitních branách před zásadními kroky, nikoli v mikromanagementu každé agentní iterace.

4.2.7 Plugins

Plugin je programové rozšíření běžící přímo v prostředí harnessu, které může doplnit exekuční jádro o systémové adaptéry, ovladače nástrojů nebo deterministické záchytné body. Plugin patří do exekuční vrstvy harnessu a používá se tehdy, když rozšíření vyžaduje programové chování namísto pouhých instrukcí pro model.

4.2.8 ReAct Loop

Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (*Reasoning + Acting*), v němž model střídá rozhodování, volání nástrojů a vyhodnocování výsledků z běhového prostředí. Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

V každé iteraci agentní smyčky harness zajišťuje tyto klíčové funkce:

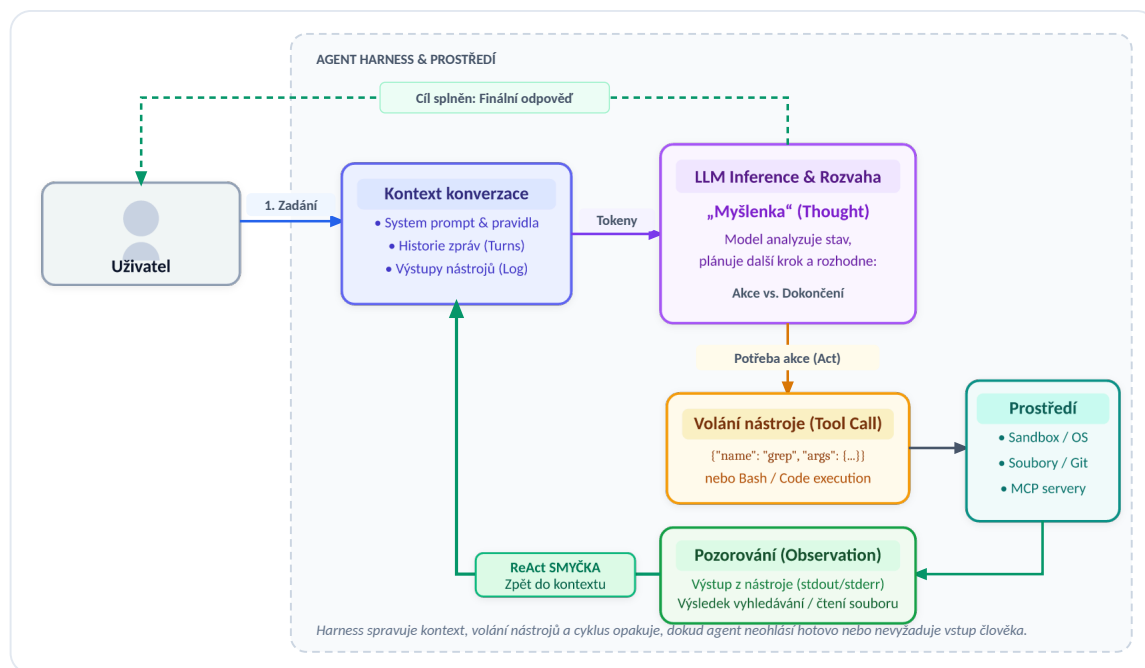
- Inicializace a správa sezení: Sestavení systémového promptu, dynamická injekce kontextu repozitáře a sledování spotřeby tokenů.
- Běhové prostředí nástrojů: Bezpečné spouštění příkazů v operačním systému a zpětné předávání výstupů modelu.

- Řízení stavových přechodů a vynucování mantinelů (Loop Engineering): Dohled nad dodržováním procesních pravidel, detekce a zastavení uvíznutých běhů a vynucování lidských schvalovacích bran.

Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*) ⁽¹²⁾, který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 9:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.



Obrázek 9: Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Agentní smyčka koordinuje opakované rozhodování modelu s vykonáváním akcí; její provozní spolehlivost proto závisí na řízení stavu, nástrojů, rozpočtů a podmínek ukončení.

4.2.8.1 Agent Session

Agentní sezení je ohraničený běh agenta se sdíleným stavem, systémovými instrukcemi, pracovním kontextem, historií tahů a provozními rozpočty. Sezení poskytuje jednotku

životního cyklu, ve které lze konzistentně spravovat stav, kontext a zdrojové limity jednoho agentního běhu.

4.2.8.2 Execution-loop Engineering

Inženýrství prováděcí smyčky je návrh a řízení stavových přechodů, podmínek ukončení, rozpočtů, opakování, eskalací a vazby mezi rozhodováním modelu a nástroji. Loop engineering převádí otevřenou agentní iteraci na provozně ohraničený proces s explicitními pravidly pokračování, selhání a ukončení.

4.2.8.3 Agent Divergence Pathologies

Divergence je třída selhání agentní smyčky, při níž se iterativní běh vzdaluje cíli například perseverací, oscilací nebo nekontrolovanou spotřebou zdrojů. Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním. V důsledku autoregresivní povahy se v kontextu snadno vytvoří pravděpodobnostní atraktor, který model uvězní v neproduktivním cyklu.

Mezi typické patologie patří:

- Perseverace a zacyklení: Opakované volání identického nástroje se stejnými neplatnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy.
- Oscilace a těkání (*Thrashing*): Střídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije modul A).
- Nekontrolovaná spotřeba zdrojů (*Context Runaway*): Rychlé vyčerpání kontextového okna i finančního rozpočtu na volání API bez dosažení cíle.

Divergenci nelze řešit pouze lepším promptem; harness musí rozpoznávat neproduktivní trajektorie a omezovat jejich pokračování.

4.2.9 Tool Calling

Vyvolávání nástrojů je mechanismus, kterým model strukturovaně žádá harness o provedení externí akce nebo funkce s validovanými parametry. Tool calling tvoří typované rozhraní mezi pravděpodobnostním rozhodnutím modelu a deterministickým vykonáním akce; formální schémata zvyšují validovatelnost za cenu kontextové režie.

4.2.9.1 Model Context Protocol

Model Context Protocol (MCP) je otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC. Pro sjednocení rozhraní mezi AI aplikacemi a externími nástroji či datovými zdroji vznikl otevřený standard **MCP** — ⁽¹³⁾. Namísto vytváření proprietárních rozhraní pro každou službu definuje MCP standardizovaný způsob komunikace.

Základní vlastnosti protokolu MCP:

- Protokolové rozhraní: Komunikace probíhá prostřednictvím standardu JSON-RPC (přes standardní vstup/výstup `stdio` nebo proud událostí `Server-Sent Events / SSE`).

- Architektonické oddělení: Implementace nástrojů běží jako samostatný proces mimo jádro harnessu. MCP servery fungují jako znovupoužitelné komponenty, které lze snadno sdílet napříč různými agenty a projekty. MCP odděluje implementaci integrací od jádra harnessu a umožňuje stejné nástroje a datové zdroje znovu používat napříč agentními systémy.

4.2.10 Skills

Dovednost je znovupoužitelný modulární balíček instrukcí, procedurálních pravidel a volitelných skriptů či zdrojů, který harness načítá podle povahy řešeného úkolu. Se vzrůstající komplexitou úloh nelze veškeré instrukce, skripty a doménové znalosti vkládat do základního systémového promptu. K modulárnímu rozšíření schopností agenta slouží **Skills**.

Architektura dovedností staví na následujících principech:

- Definiční soubor `SKILL.md`: Dovednost tvoří adresář obsahující definiční soubor se strukturovanou hlavičkou (YAML frontmatter vymezující název a popis role) a detailním návodem k použití.
- Dynamické načítání pro úsporu kontextu: Do výchozího promptu se vloží pouze stručný přehled dostupných dovedností. Kompletní instrukce a skripty se do kontextu načtou až v okamžiku, kdy agent danou dovednost explicitně vyvolá.
- Skripty () a záchytné body (Hook): Dovednosti mohou obsahovat deterministické skripty pro rutinní transformace kódu a událostní háčky vyvolávané při stavových přechodech harnessu.

Kromě kontextových dovedností využívají pokročilé řídicí architektury také programové **Plugins** — . Zatímco *Skills* fungují jako kontextové procedury a instrukce interpretované modelem, pluginy rozšiřují samotný harness na nativní systémové úrovni. Dovednosti modularizují doménové postupy a načítají je pouze tehdy, když jsou relevantní, čímž snižují velikost základního promptu a omezují duplicitu instrukcí.

4.2.10.1 Progressive Disclosure

Postupné zpřístupňování je strategie, při níž se do aktivního kontextu nejprve vkládají pouze stručné popisy schopností a podrobné instrukce se načtou až při jejich použití. Do aktivního kontextu se dostávají pouze právě potřebné instrukce, čímž se omezuje režie bez ztráty dostupnosti specializovaných postupů.

4.2.10.2 Script

Skript je soubor nebo posloupnost příkazů určených k automatizovanému vykonání interpretem, shellem nebo jiným běhovým prostředím. Skripty přesouvají opakovatelné deterministické kroky mimo pravděpodobnostní rozhodování modelu.

4.2.10.3 Event Hook

Hook je definovaný bod životního cyklu nebo události, na který lze navázat vlastní deterministickou logiku před, po nebo místo standardního chování systému. Hook umožňuje

vynutit opakovatelné chování v přesně určeném okamžiku bez toho, aby model musel stejné pravidlo znovu odvozovat.

4.2.11 Context Engineering

Kontextové inženýrství je systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu. Kontext zahrnuje systémové instrukce, pracovní historii, výsledky nástrojů, externě načtená data a další informace, které model používá při rozhodování. Návrh této vrstvy určuje, které informace jsou modelu dostupné, kdy se načítají, jak dlouho zůstávají aktivní a jak se nahrazují při omezené kapacitě kontextového okna. Kontextové inženýrství řídí informační prostředí modelu; cílem není maximalizovat objem kontextu, ale udržet relevantní, přesný a provozně použitelný stav.

4.2.11.1 Context Compaction

Kompakce kontextu je zmenšení aktivního kontextu výběrem, shrnutím nebo nahrazením starší historie kompaktnější reprezentací tak, aby se běh vešel do kontextového okna. Správa aktivního kontextu je součástí kontextového inženýrství (). Při rozsáhlejších úlohách se kontextové okno nevyhnutelně zaplní. V okamžiku, kdy objem historie dosáhne kritické hranice, musí agent harness přistoupit ke kompakci kontextu (*compaction*) — model je vyzván, aby dosavadní průběh sezení zkrátil do syntetického souhrnu, který nahradí starší část historie.

Tento proces však představuje destruktivní ztrátovou kompresi:

- Ztráta deterministických detailů: Model při rekurzivním zkracování vynechává přesná čísla řádků, signatury privátních funkcí, přesné cesty k souborům a doslovná chybová hlášení kompilátoru.
- Oslabení negativních pravidel: Explicitní zákazy (např. neměnit veřejné rozhraní API) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.
- Konfirmační zkreslení (*Confirmation Bias*): Model v souhrnu upřednostňuje fakta odpovídající jeho vnitřním statistickým asociacím na úkor netriviálních specifik konkrétního projektu.

Kompakce prodlužuje běh za cenu ztráty informace, a proto musí být navržena tak, aby neodstraňovala detaily nutné pro další rozhodování.

4.2.11.2 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) je architektura, v níž systém před generováním nebo během něj vyhledá relevantní informace z externího zdroje a vloží je do kontextu modelu. Aby se předešlo ztrátě informací způsobené kompakcí, moderní agentní architektury přesouvají část paměti mimo samotné kontextové okno. Namísto spoléhání se na jediný lineární textový kontext se uplatňují strukturovaná externí úložiště.

K hlavním přístupům patří:

- Hierarchická epizodická paměť (RAG): Ukládání doslovných protokolů nástrojů a historie úloh do externí databáze; do kontextu se selektivně injektují pouze bezprostředně relevantní fragmenty.
 - Persistentní graf stavu projektu (*Project State Graph*): Udržování explicitního, strukturovaného přehledu o stavu repozitáře (seznam modifikovaných souborů, otevřené úkoly, výsledky testů a platné invarianty) mimo kontextové okno.
- Díky tomu může agent kdykoliv obnovit přesný stav projektu bez závislosti na ztrátovém rekurzivním shrnování. RAG odděluje dlouhodobé uchování informací od omezeného aktivního kontextu a umožňuje načítat pouze data relevantní pro aktuální krok.

4.2.11.3 Context Rot

Degradace kontextu je pokles schopnosti modelu spolehlivě využívat informace v dlouhém, hlučném nebo vzájemně si konkurujícím kontextu. Schopnost jazykového modelu pracovat s dlouhým kontextem nelze posuzovat pouze podle nominální velikosti okna. Ačkoliv moderní modely deklarují kapacitu statisíců tokenů, jejich schopnost efektivně vyhledávat a logicky propojovat fakta s rostoucí délkou kontextu výrazně klesá. Tento jev se označuje jako *Context Rot*.

V praxi se projevuje dvěma hlavními mechanismy:

- *Lost in the Middle* ⁽¹⁴⁾: Pozornostní vrstvy transformeru spolehlivě vnímají informace na samém začátku a konci okna, zatímco fakta umístěná uprostřed dlouhého textu jsou často přehlížena.
- *Multi-Needle Reasoning*: Schopnost logicky provázat několik na sobě závislých informací rozptýlených napříč různými soubory; s rostoucí délkou kontextu tato schopnost prudce klesá.

Při komplexním křížovém refaktoringu ve velkém kontextu proto model často přehlédne klíčové souvislosti, které by v menším a čistším okně zpracoval bez potíží. Nominálně větší kontextové okno samo o sobě nezaručuje lepší výkon; kvalita závisí na tom, zda model dokáže relevantní informace v kontextu skutečně najít a propojit.

4.2.11.4 Semantic Drift

Sémantický posun je postupné zkreslování významu a faktického stavu při opakovaném ztrátovém shrnování nebo transformaci kontextu. Pokud je historie sezení v dlouhém vývojovém běhu shrnována vícekrát po sobě, vzniká řetězec ztrátových transformací ($S_{k+1} = f(S_k, \Delta_k)$).

Rizika sémantického posunu spočívají v těchto jevech:

- *Efekt tiché pošty*: Drobné zkreslení či halucinace vzniklá v kole k je v kole $k + 1$ přijata jako nezpochybnitelný historický fakt.
- *Divergence modelu od reality*: Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému.

Výsledkem je stav, kdy agent sebevědomě reportuje vyřešení úkolu, ačkoliv reálný kód zůstává v nefunkčním či neúplném stavu. Opakovaná komprese může změnit pracovní

reprezentaci reality natolik, že další rozhodování vychází z chybných historických předpokladů.

4.2.12 Workflow-graph Engineering

Inženýrství pracovních grafů je návrh agentních nebo automatizačních workflow jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky. Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

Grafová struktura umožňuje rozdělit složitou úlohu na kontrolovatelné kroky, oddělit jejich odpovědnosti a explicitně řídit závislosti mezi nimi.

4.2.12.1 Subagent

Subagent je dočasná nebo specializovaná agentní instance, které nadřazený orchestrátor deleguje vymezenou dílčí úlohu a následně převezme její výsledek. Subagenti umožňují paralelní nebo specializovanou práci a současně omezují množství dílčí pracovní historie, které musí zůstat v kontextu hlavního orchestrátoru.

4.2.12.2 Directed Acyclic Graph

Orientovaný acyklický graf (DAG) je orientovaný graf bez orientovaného cyklu, který umožňuje explicitně vyjádřit závislosti a pořadí kroků pracovního postupu. DAG převádí implicitní pořadí vícefázové úlohy na kontrolovatelnou strukturu závislostí, v níž lze navazující krok spustit až po splnění jeho předpokladů.

5 Results and Discussion

6 Conclusion

List of figures and tables

Obrázek 1	AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute.	8
Obrázek 2	Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.	14
Obrázek 3	GPT-5.6. Zdroj: OpenAI.	15
Obrázek 4	Claude Opus 5. Zdroj: Anthropic.	16
Obrázek 5	DeepSeek-V4.1-Flash. Zdroj: DeepSeek.	16
Obrázek 6	Aplikace Codex. Zdroj: OpenAI.	20
Obrázek 7	Claude Code při práci se souborem. Zdroj: Anthropic.	21
Obrázek 8	Aplikační rozhraní Claude. Zdroj: Anthropic.	22
Obrázek 9	Architektura autonomní ReAct smyčky (Reasoning + Acting) a tok dat mezi uživatelé, kontextem, modelem a výkonným prostředím.	24

List of appendices

References

1. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion: Q1 2026 Trends and Insights. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
2. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
3. ANTHROPIC. Claude Code. Online. 2026. [Accessed 20 září 2026]. Available from: <https://claude.com/product/claude-code>
4. MICROSOFT AI ECONOMY INSTITUTE. Global AI Adoption in 2025—A Widening Digital Divide. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/corporate-responsibility/topics/ai-economy-institute/reports/global-ai-adoption-2025/>
5. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
6. OPENAI. ChatGPT macOS app—Screenshot Tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://help.openai.com/en/articles/9295245-chatgpt-macos-app-screenshot-tool>
7. OPENAI. GPT-5.6: Frontier intelligence that scales with your ambition. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/gpt-5-6/>
8. ANTHROPIC. Introducing Claude Opus 5. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.anthropic.com/news/claude-opus-5>
9. DEEPSEEK. Introducing DeepSeek-V4.1-Flash: smarter, faster, more efficient. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.deepseek.com/en/news/deepseek-v4-1-flash/>
10. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
11. ANTHROPIC. Navigating the Claude desktop app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://academy.claude.com/tutorials/navigating-the-claude-desktop-app>
12. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
13. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>

14. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
15. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
16. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
17. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
18. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
19. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
20. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
21. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.
22. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
23. SCHICK, Timo, DWIVEDI-YU, Jane, DESSÌ, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
24. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄ-

- SCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
25. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
 26. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
 27. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
 28. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
 29. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
 30. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
 31. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
 32. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
 33. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
 34. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
 35. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Procee-*

- dings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
36. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.